

S.Y.B.Sc.(Data Science)

Sem- IV

Subject: Data Structure-II

Unit1 : Efficient Search Tree

By,

Mrs. Reshma Masurekar

*Dr. D. Y. Patil Arts, Commerce and Science College,
Pimpri, Pune 18*

Need of Height Balanced Tree?

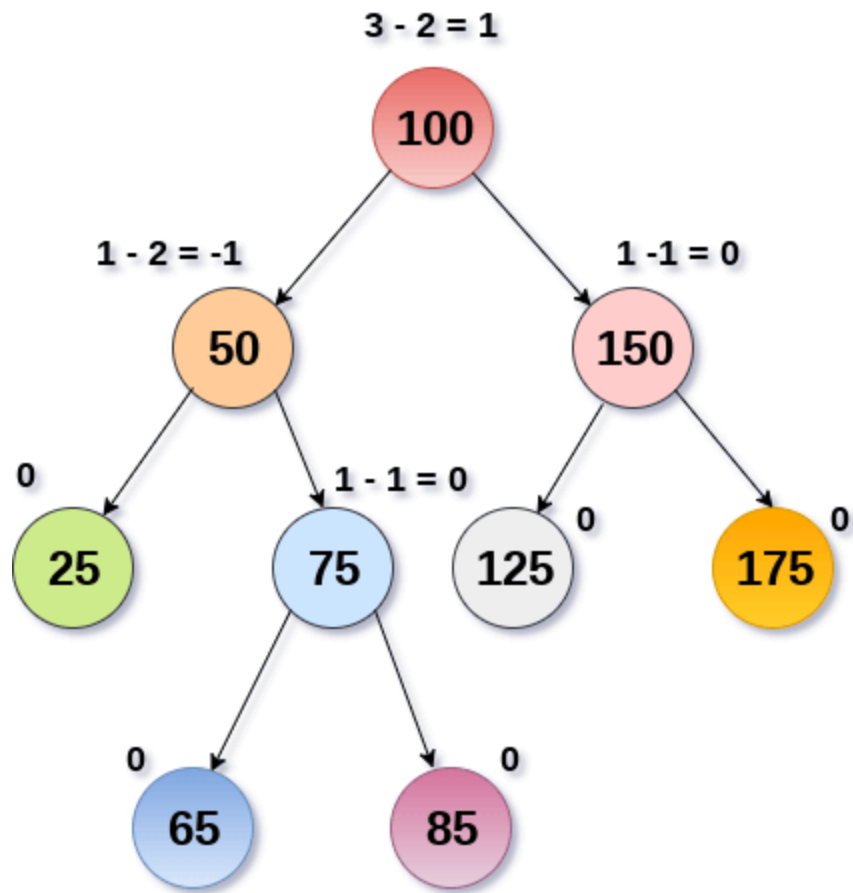
- In case of binary trees, if the trees are skewed, they become computationally inefficient to perform operations on.
- This is the motivation behind making sure that trees are not skewed. Hence the need for balanced binary trees.

Height Balanced tree

- A height balanced binary tree is a binary tree in which the height of the left subtree and right subtree of any node does not differ by more than 1 and both the left and right subtree are also height balanced.
- Balanced Binary trees are computationally efficient to perform operations.
- A balanced binary tree will follow the following conditions:
 1. The absolute difference of heights of left and right subtrees at any node is less than 1.
 2. For each node, its left subtree is a balanced binary tree.
 3. For each node, its right subtree is a balanced binary tree.

AVL Tree

- AVL Tree is invented by GM Adelson - Velsky and EM Landis in 1962.
- The tree is named AVL in honour of its inventors.
- AVL Tree is called as height balanced binary search tree in which each node is associated with a balance factor which is calculated by subtracting the height of its right sub-tree from that of its left sub-tree.
- **Tree is called as balanced** if balance factor of each node is in between -1 to 1, otherwise, the tree will be unbalanced and need to be balanced.
- **Balance Factor (k)** = height (left(k)) - height (right(k))



AVL Tree

Complexity

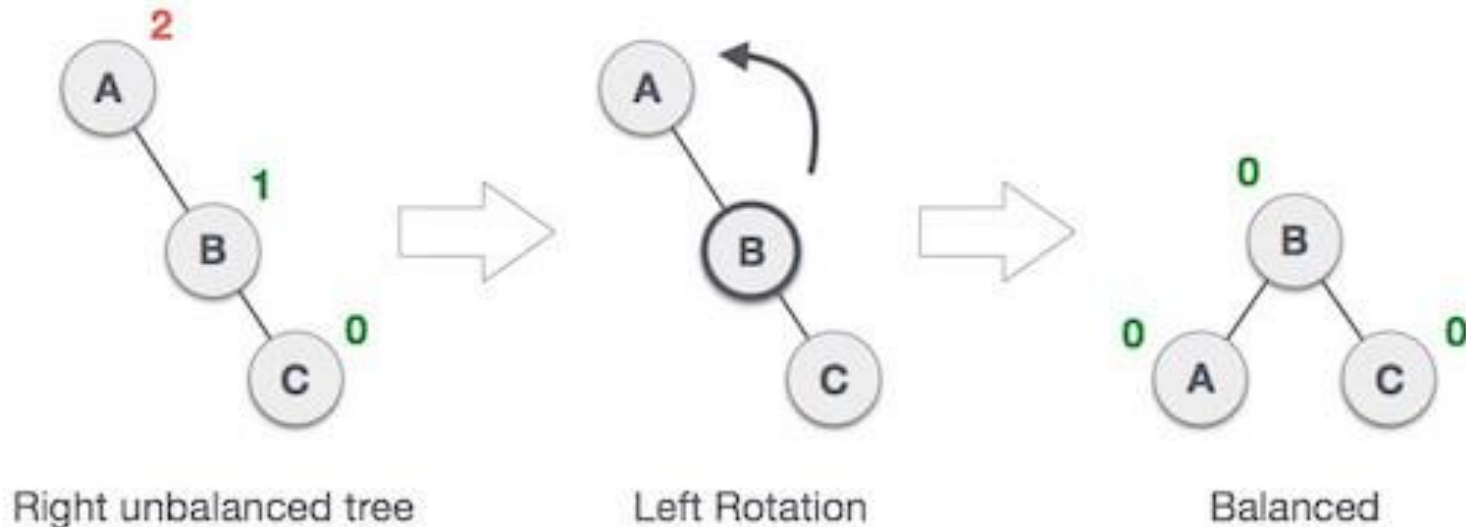
Algorithm	Average case	Worst case
Space	$O(n)$	$O(n)$
Search	$O(\log n)$	$O(\log n)$
Insert	$O(\log n)$	$O(\log n)$
Delete	$O(\log n)$	$O(\log n)$

AVL Rotations

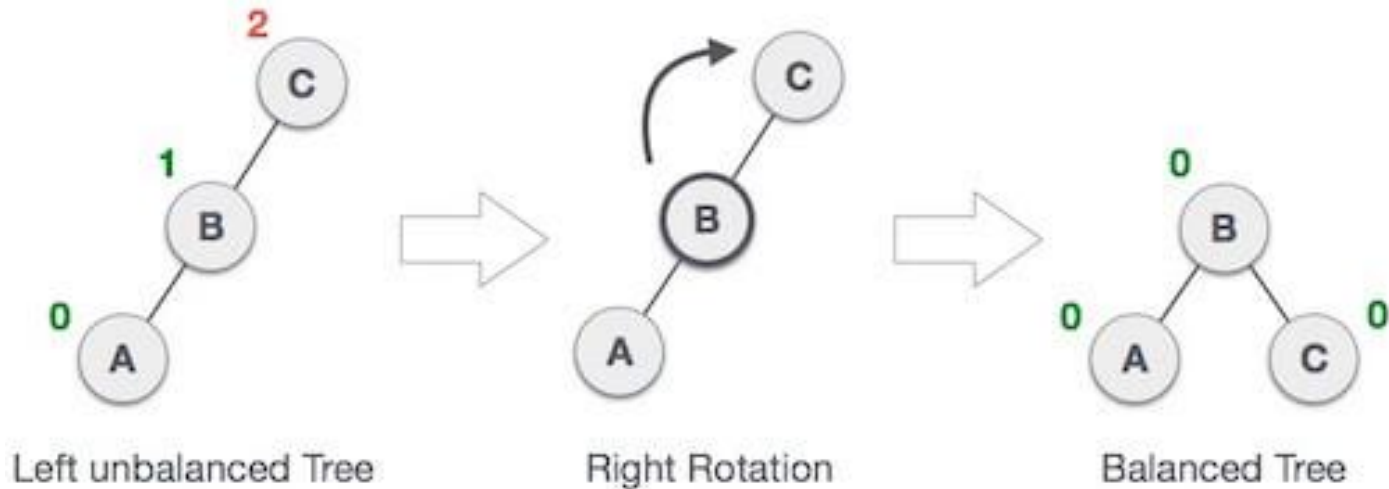
- We perform rotation in AVL tree only in case if Balance Factor is other than **-1, 0, and 1**. There are basically four types of rotations which are as follows:
- **LL rotation:** Inserted node is in the left subtree of left subtree of A
- **RR rotation:** Inserted node is in the right subtree of right subtree of A
- **LR rotation:** Inserted node is in the right subtree of left subtree of A
- **RL rotation:** Inserted node is in the left subtree of right subtree of A
- Where node A is the node whose balance Factor is other than -1, 0, 1.

- If balance factor of any node is **1**, it means that the left sub-tree is one level higher than the right sub-tree.
- If balance factor of any node is **0**, it means that the left sub-tree and right sub-tree contain equal height.
- If balance factor of any node is **-1**, it means that the left sub-tree is one level lower than the right sub-tree.

- **RR rotation:** When BST becomes unbalanced, due to a node is inserted into the right subtree of the right subtree of A, then RR rotation is performed.
- RR rotation is an anticlockwise rotation, which is applied on the edge below a node having balance factor -2.
- **Example,** node A has balance factor -2 because a node C is inserted in the right subtree of A right subtree. So perform the RR rotation on the edge below A.



- **LL rotation:** When BST becomes unbalanced, due to a node is inserted into the left subtree of the left subtree of C, LL rotation is performed.
- LL rotation is clockwise rotation, which is applied on the edge below a node having balance factor 2.
- **Example,** node C has balance factor 2 because a node A is inserted in the left subtree of C left subtree. So perform the LL rotation on the edge below A.

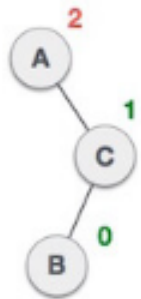
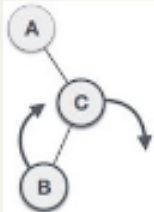
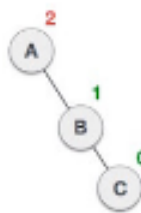
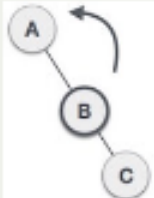
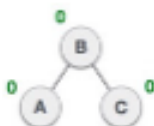


- **LR Rotation (Left-Right):**
- **Occurs when** a node is inserted into the right subtree of the left child, which disturbs the balance factor of an ancestor node, making it **left-heavy**.
- LR rotation = RR rotation + LL rotation, i.e., first RR rotation is performed on subtree and then LL rotation is performed on full tree, by full tree i.e the first node from the path of inserted node whose balance factor is other than -1, 0, or 1.
- How it works: A double rotation is performed.
 - **Right rotation:** Perform a right rotation on the left child (B) of the unbalanced node (A).
 - **Left rotation:** Then, perform a left rotation on the original unbalanced node (A).

State	Action
	A node B has been inserted into the right subtree of A the left subtree of C, because of which C has become an unbalanced node having balance factor 2. This case is L R rotation where: Inserted node is in the right subtree of left subtree of C
	As LR rotation = RR + LL rotation, hence RR (anticlockwise) on subtree rooted at A is performed first. By doing RR rotation, node A, has become the left subtree of B.
	After performing RR rotation, node C is still unbalanced, i.e., having balance factor 2, as inserted node A is in the left of left of C
	Now we perform LL clockwise rotation on full tree, i.e. on node C. node C has now become the right subtree of node B, A is left subtree of B
	Balance factor of each node is now either -1, 0, or 1, i.e. BST is balanced now.

- **RL Rotation:**

- Occurs when a node is inserted into the left subtree of the right child, which disturbs the balance factor of an ancestor node, making it **right-heavy**.
- RL rotation = LL rotation + RR rotation, i.e., first LL rotation is performed on subtree and then RR rotation is performed on full tree, by full tree i.e the first node from the path of inserted node whose balance factor is other than -1, 0, or 1.
- **How it works:** A double rotation is performed.
 - **Left Rotation:** Perform a left rotation on the right child (C) of the unbalanced node (A).
 - **Right Rotation:** Then, perform a right rotation on the original unbalanced node (A).

State	Action
	A node B has been inserted into the left subtree of C the right subtree of A, because of which A has become an unbalanced node having balance factor - 2. This case is RL rotation where: Inserted node is in the left subtree of right subtree of A
	As RL rotation = LL rotation + RR rotation, hence, LL (clockwise) on subtree rooted at C is performed first. By doing RR rotation, node C has become the right subtree of B.
	After performing LL rotation, node A is still unbalanced, i.e. having balance factor -2, which is because of the right-subtree of the right-subtree node A.
	Now we perform RR rotation (anticlockwise rotation) on full tree, i.e. on node A, node C has now become the right subtree of node B, and node A has become the left subtree of B.
	Balance factor of each node is now either -1, 0, or 1, i.e., BST is balanced now.